

Contents

Dynamic Programming — Knapsack (0/1 & Unbounded) — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	9
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	10
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	14
SECTION 17 — Multiple Language Templates	14
SECTION 18 — Dry Runs	16
SECTION 19 — Advanced Concepts	16
SECTION 20 — Senior Engineer Insights	17
SECTION 21 — Cheat Sheet	17
SECTION 22 — Revision Notes (5-Minute Review)	17
SECTION 23 — Practice Roadmap	18
SECTION 24 — Memory Tricks	18
SECTION 25 — Final Summary	18

Dynamic Programming — Knapsack (0/1 & Unbounded) — Complete Interview Handbook

Pattern #24 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — DP on Subsequences / Knapsack section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Knapsack DP solves problems where you must **choose a subset of items, each with a weight/cost and a value, to maximize/minimize/count something subject to a capacity constraint**. $dp[i][w]$ represents “the best achievable value using only the first i items, with capacity w remaining.” **0/1 Knapsack** allows each item to be used **at most once**; **Unbounded Knapsack** allows each item to be used **unlimited times** — this single distinction changes the loop structure and is the most commonly tested subtlety in this pattern.

1.2 Why Was This Pattern Invented?

“Which subset of items should I pick, given a budget/capacity constraint, to maximize value?” is an extremely common real-world question (resource allocation, budgeting, packing) that’s NP-hard to solve exactly by brute-force subset enumeration (2^n subsets). Knapsack DP exploits the fact that the decision for item i (include or exclude) only depends on the **remaining capacity**, not on which specific earlier items were chosen — collapsing the state space from “which subset” (exponential) to “how many items considered, how much capacity remains” (polynomial, $O(n \times \text{capacity})$).

1.3 Real Intuition Behind The Pattern

Imagine **packing a suitcase with a strict weight limit**, choosing among items each with a weight and a “usefulness” score, to maximize total usefulness without exceeding the limit. For each item, you face a binary choice: take it (using up some capacity, gaining its value) or leave it (capacity unchanged) — and critically, **the best decision for the remaining items only depends on how much capacity is left**, not on which specific items filled the used capacity.

1.4 Mental Model

For each item, at each possible capacity: “what’s better — skipping this item ($dp[i-1][w]$), or taking it (item’s value + $dp[i-1][w - \text{item's weight}]$)?” The 0/1 vs. unbounded distinction is entirely about whether “taking it” references the **previous item row** (0/1 — each item used once) or the **same item row** (unbounded — item can be reused).

1.5 Visual Explanation

0/1 Knapsack: items = [(weight=1,value=1), (weight=3,value=4), (weight=4,value=5)], capacity=4

$dp[i][w]$ = max value using first i items, capacity w

	w=0	w=1	w=2	w=3	w=4	
i=0:	0	0	0	0	0	
i=1:	0	1	1	1	1	(item1: w=1,v=1)
i=2:	0	1	1	4	5	(item2: w=3,v=4 → $dp[2][4] = \max(dp[1][4]=1, 4+dp[1][1]=4+1=5)=5$)
i=3:	0	1	1	4	5	(item3: w=4,v=5 → $dp[3][4] = \max(dp[2][4]=5, 5+dp[2][0]=5+0=5)=5$)

Answer: $dp[3][4] = 5$

1.6 Simple Analogy

0/1 Knapsack is like **shopping with a fixed budget where each unique item is only available once** — buy it or don’t, but you can’t buy two of the same item. Unbounded Knapsack is like **shopping at a store with unlimited stock of each item** — you can buy as many of the same item as your budget allows (like making change with unlimited coins of each denomination).

1.7 When Should I Immediately Think About Using This Pattern?

- “Maximize value subject to a weight/capacity constraint” with items usable **once**.
- “Minimum number of coins to make change” or “number of ways to make change” — Unbounded Knapsack (unlimited coin reuse).
- “Partition into two equal-sum subsets,” “subset sum equals target” — 0/1 Knapsack (existence variant).
- “Number of ways to fill a knapsack” (counting variant, not just optimization).

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“maximize value, weight limit”	Classic 0/1 Knapsack
“subset sum,” “partition equal subset sum”	0/1 Knapsack (existence)
“coin change,” “minimum coins,” “ways to make change”	Unbounded Knapsack
“unlimited supply,” “as many as you want”	Unbounded Knapsack signal
“each item used at most once”	0/1 Knapsack signal
“target sum,” “count of ways to reach target”	0/1 or Unbounded Knapsack counting variant

2.2 Hidden Hints

The phrase “**at most once**” vs. “**unlimited**”/“**as many times as needed**” is the single most important distinguishing detail — always read the problem statement specifically for this, since it changes the loop order/direction entirely.

2.3 Interview Clues

Interviewer emphasizes a **capacity/budget/target constraint** alongside a set of items each with a cost/weight — this two-dimensional constraint (items AND capacity) is the definitive Knapsack signal.

2.4 Common Trick Words

“Fill exactly,” “at most,” “at least” (subtly changes whether unreachable states should be treated as 0, -infinity, or invalid, depending on whether over/under-filling is allowed).

2.5 What Interviewers Expect

Correct 0/1 vs. Unbounded distinction (and the corresponding loop-order/space-optimization implications — see §16), correct space optimization to $O(\text{capacity})$ using a 1D rolling array, and correct handling of “exactly,” “at most,” and “at least” capacity semantics.

2.6 When NOT To Use This Pattern

- There’s **no capacity/weight constraint** at all — this degenerates to a simpler problem (perhaps plain 1D DP or Greedy) without the two-dimensional item+capacity state.
- You need to enumerate **all** subsets satisfying the constraint, not just the optimal value/count — that’s Backtracking (Pattern #12), though Knapsack DP can sometimes be extended to reconstruct one such subset.

- The “capacity” is actually enormous (e.g., 10^{18}) and items are few — then Knapsack DP’s $O(n \times \text{capacity})$ becomes infeasible, and a different approach (meet-in-the-middle for small n , or a completely different technique) is needed.

SECTION 3 — Decision Framework

Does the problem involve CHOOSING a subset of items under a CAPACITY/WEIGHT/BUDGET constraint?

Yes

Can each item be used AT MOST ONCE?

Yes → USE 0/1 KNAPSACK (iterate capacity BACKWARD when space-optimizing to 1D)

No - can items be reused UNLIMITED times?

Yes → USE UNBOUNDED KNAPSACK (iterate capacity FORWARD when space-optimizing to 1D)

Do you need the OPTIMAL VALUE, EXISTENCE (can you reach target?), or COUNT (number of ways)?

Value → max/min combine function

Existence → boolean OR combine function

Count → sum combine function

Is the capacity astronomically large relative to the number of items?

Yes → Knapsack DP’s $O(n \times \text{capacity})$ is infeasible - reconsider (meet-in-the-middle, different algo)

Why: The entire 0/1 vs. Unbounded distinction reduces to one implementation detail — the direction of the capacity loop when space-optimized to 1D — but getting this backward silently produces wrong answers (allowing item reuse when it shouldn’t be allowed, or vice versa), making it the single most tested subtlety in this pattern.

SECTION 4 — Why This Pattern Works

Mathematical/Logical (State Space Collapse): Brute-force subset enumeration considers 2^n possible subsets. Knapsack DP’s key insight is that the value of the *optimal* solution using items $1..i$ with capacity w depends **only** on i and w — not on which specific combination of earlier items was used to arrive at that remaining capacity. This collapses the state space from “which exact subset” (2^n possibilities) to “(items considered, capacity remaining)” ($n \times \text{capacity}$ possibilities), a polynomial (pseudo-polynomial, since it depends on the numeric value of capacity, not just input size) reduction.

Correctness Proof (0/1 Knapsack, by induction on i): *Base case:* $dp[0][w] = 0$ for all w (no items considered, no value achievable) — trivially correct. *Inductive hypothesis:* assume $dp[i-1][w]$ is correct for all w . *Inductive step:* for item i with weight wt and value val , $dp[i][w] = \max(dp[i-1][w], val + dp[i-1][w-wt])$ (if $w \geq wt$) correctly considers both possibilities — excluding item i (using the already-correct $dp[i-1][w]$) or including it (using the already-correct $dp[i-1][w-wt]$, since item i can only be used once, so the “remaining budget after taking it” must be solved using only items $1..i-1$). Taking the max of these two exhaustively covers both cases

without missing or double-counting any valid selection. *Termination:* $dp[n][capacity]$ is correct by induction. **QED.**

Why Unbounded Differs: Since an item can be reused, “including item i ” should reference $dp[i][w-wt]$ (the **same** item row, allowing further reuse of item i), not $dp[i-1][w-wt]$ — this single change in the recurrence (and correspondingly, the loop direction when space-optimized) is what permits unlimited reuse while 0/1 Knapsack’s $dp[i-1][...]$ reference prevents it.

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (0/1 Knapsack)

1. Define $dp[i][w]$ = best value using the first i items with capacity w .
2. Base case: $dp[0][w] = 0$ for all w (no items, no value).
3. Recurrence: $dp[i][w] = \max(dp[i-1][w], value[i] + dp[i-1][w - weight[i]])$ if $weight[i] \leq w$, else $dp[i][w] = dp[i-1][w]$.
4. Answer: $dp[n][capacity]$.

5.2 General Template — 0/1 Knapsack (2D)

```
function knapsack01(weights, values, capacity, n):
    dp = 2D array of size (n+1) x (capacity+1), all zeros

    for i in range(1, n+1):
        for w in range(0, capacity+1):
            dp[i][w] = dp[i-1][w]                # exclude item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], values[i-1] + dp[i-1][w - weights[i-1]]) # include item i

    return dp[n][capacity]
```

5.3 Space-Optimized Template — 0/1 Knapsack (1D, capacity loop BACKWARD)

```
function knapsack01Optimized(weights, values, capacity, n):
    dp = array of size capacity+1, all zeros

    for i in range(0, n):
        for w in range(capacity, weights[i] - 1, -1): # BACKWARD - prevents reusing item i
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[capacity]
```

5.4 Space-Optimized Template — Unbounded Knapsack (1D, capacity loop FORWARD)

```
function unboundedKnapsack(weights, values, capacity, n):
    dp = array of size capacity+1, all zeros

    for i in range(0, n):
        for w in range(weights[i], capacity+1): # FORWARD - allows reusing item i
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[capacity]
```

5.5 Interview Thinking Process

1. "This is a subset-selection problem under a capacity constraint — I'll use Knapsack DP with state (items considered, capacity remaining)."
 2. "I need to check: can each item be used at most once (0/1) or unlimited times (Unbounded)? This determines my loop direction when space-optimizing."
 3. "I'll define the combine function based on what's asked: max/min for optimal value, OR for existence, sum for counting ways."
 4. "I'll space-optimize to a 1D array of size (capacity+1), iterating capacity backward for 0/1 and forward for Unbounded."
-

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n \times \text{capacity})$	$O(n \times \text{capacity})$, reducible to $O(\text{capacity})$ with 1D rolling array	Each of n items considered against each of capacity possible remaining budgets
Average Case	$O(n \times \text{capacity})$	$O(\text{capacity})$ optimized	Deterministic per-cell cost, no data-dependent variation
Best Case	$O(n \times \text{capacity})$ (must fill the full table in general)	$O(\text{capacity})$ optimized	Even simple inputs require the full DP table computation
Amortized	$O(n \times \text{capacity})$ total — pseudo-polynomial (depends on capacity's numeric value, not just n)	$O(\text{capacity})$	This is "pseudo-polynomial" specifically because capacity is a number, not the input size in the traditional sense — worth explicitly mentioning in interviews

Important complexity nuance: Knapsack DP's $O(n \times \text{capacity})$ is called **pseudo-polynomial** because capacity can be exponentially large relative to the number of bits needed to represent it — this is why 0/1 Knapsack is NP-hard in the strict sense (no truly polynomial algorithm is known), even though this DP approach is efficient for reasonably-sized numeric capacities.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
capacity = 0	No room for any item	$\text{dp}[i][0] = 0$ for all i — correctly no value achievable

Edge Case	Example	Handling
Single item, weight exceeds capacity	Item can't fit at all	Correctly excluded — $dp[n][capacity]$ equals $dp[n-1][capacity]$
All items have weight 0 (edge case in some framings)	Zero-weight items	Must clarify problem semantics — potentially all such items can always be included
$n = 0$ (no items)	Empty item set	$dp[0][w] = 0$ for all w — trivially no value achievable
Target sum unreachable (Subset Sum / Partition Equal Subset Sum)	No valid subset sums to target	Existence-variant DP correctly returns false
Coin Change with no valid combination	Target amount unreachable with given coins	Must return a sentinel (e.g., -1) distinct from “0 coins needed,” requiring careful initialization (often initialize to infinity, check if still infinity at the end)
Very large capacity (pseudo-polynomial blowup)	$capacity = 10^9$	The $O(n \times capacity)$ approach becomes infeasible — must recognize this limit and discuss alternatives

Common mistakes: using the wrong loop direction for 0/1 vs Unbounded when space-optimizing (this is THE classic Knapsack bug — allowing/disallowing reuse incorrectly); forgetting to distinguish “0 ways/0 value achievable” from “target unreachable” in existence/counting variants (often requiring different sentinel values, like -1 or infinity, rather than defaulting to 0).

SECTION 8 — Pros & Cons

Advantages: Converts exponential subset enumeration (2^n) into pseudo-polynomial $O(n \times capacity)$ time; the 0/1 vs Unbounded distinction elegantly captures a huge family of real-world “selection under constraint” problems with a single small implementation change. **Disadvantages:** Pseudo-polynomial complexity means very large numeric capacities make this approach infeasible, even for small n ; the 0/1 vs Unbounded loop-direction distinction is subtle and a common source of bugs. **Trade-offs:** 2D table (easier to reason about, supports path reconstruction) vs. 1D space-optimized ($O(capacity)$ space, but loop direction becomes critical and path reconstruction requires extra bookkeeping). **Limitations:** NP-hard in the strict theoretical sense (no truly polynomial algorithm exists for arbitrary-precision capacities); doesn't scale to enormous numeric capacities regardless of n . **Inefficient when:** capacity is astronomically large relative to a small number of items — reconsider meet-in-the-middle or other exponential-but-smarter techniques instead.

SECTION 9 — Real World Applications

Domain	Application
Logistics/Shipping	Cargo loading optimization — selecting which packages to load onto a truck/container with a weight limit to maximize value shipped
Finance	Portfolio selection under a budget constraint (choosing which assets to buy to maximize expected return within a budget)
Cloud Computing	Resource allocation — selecting which VM instances/tasks to run on a server with limited CPU/memory capacity to maximize throughput
Retail/E-commerce	Product bundling and promotional selection optimization under a cost constraint
Manufacturing	Cutting stock problems (minimizing waste when cutting materials into required sizes) — related Knapsack variant
Currency Systems	Coin change (minimum coins, number of ways) — Unbounded Knapsack, directly used in point-of-sale/ATM cash-dispensing logic
Project Management	Selecting which projects to fund given a budget to maximize total expected value/ROI
Genomics	Selecting a subset of genetic markers/features under a computational/storage budget to maximize predictive information
Advertising	Ad slot selection under a budget constraint to maximize expected click-through/conversion value
Data Compression	Certain resource-constrained encoding schemes use Knapsack-style optimization for bit allocation

SECTION 10 — Interview Strategy

How seniors answer: They immediately identify whether items are reusable (Unbounded) or single-use (0/1), state the correct loop direction for space optimization and *why* it matters, and clarify whether the problem needs optimal value, existence, or counting before choosing the combine function.

How juniors answer: They often get the 0/1 vs Unbounded loop direction backward when space-optimizing (a subtle, easy-to-miss bug), or they don't recognize a problem as Knapsack DP at all when it's disguised (e.g., "Partition Equal Subset Sum" doesn't mention "knapsack" explicitly).

Typical follow-ups: "What if items could be used unlimited times instead — what changes?" "Can you reconstruct which items were actually chosen, not just the optimal value?" "What if the capacity were astronomically large — does your approach still work?" "How is this different from a plain subset-sum problem?"

Optimization questions: "Can you reduce space from $O(n \times \text{capacity})$ to $O(\text{capacity})$?" (Yes — the standard 1D rolling array optimization, with the critical loop-direction distinction.)

SECTION 11 — Pattern Variations

Variation	Description	Example
0/1 Knapsack (Value Maximization)	Classic single-use item selection	0/1 Knapsack (classic, GFG/interview framing)
Subset Sum (Existence)	Can a subset sum to exactly target?	Partition Equal Subset Sum
Counting Subsets	How many subsets sum to target?	Target Sum, Partition Equal Subset Sum (count variant)
Unbounded Knapsack (Value Maximization)	Items reusable, maximize value	Rod Cutting
Coin Change (Minimum Count)	Unbounded, minimize item count	Coin Change
Coin Change (Counting Ways)	Unbounded, count combinations	Coin Change II
Bounded Knapsack (Limited Reuse Count)	Each item usable up to k times	Bounded Knapsack (less common in interviews, but a natural generalization)
Multi-Dimensional Knapsack	Two or more capacity constraints simultaneously (e.g., weight AND volume)	Ones and Zeroes (0s and 1s as dual capacity constraints)

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
1D DP	Single state dimension, no capacity constraint	No item-selection-under-constraint structure
2D Grid DP	Also 2D, but state is (position, position) not (item, capacity)	Grid/sequence-comparison problems, not selection-under-constraint
Backtracking	Enumerates all subsets exhaustively, no capacity-based state collapse	Need to enumerate actual subsets, not just optimal value/count
Greedy (Fractional Knapsack)	If items can be fractionally taken, a greedy value/weight ratio approach is optimal — but this does NOT work for 0/1 Knapsack	Items ARE divisible (fractional Knapsack), not discrete 0/1 selections

Comparison Table

Aspect	0/1 Knapsack	Unbounded Knapsack	Fractional Knapsack (Greedy)
Item reuse	Never	Unlimited	N/A (items divisible)
Optimal technique	DP (loop backward when 1D)	DP (loop forward when 1D)	Greedy (sort by value/weight ratio)
Complexity	$O(n \times \text{capacity})$	$O(n \times \text{capacity})$	$O(n \log n)$

SECTION 13 – Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct value maximization	0/1 Knapsack (classic framing, common on GFG)
Medium	Existence/counting variants, unbounded basics	Partition Equal Subset Sum, Coin Change, Coin Change II
Hard	Multi-dimensional constraints, advanced counting	Target Sum, Ones and Zeroes, Last Stone Weight II
Very Hard	Combined constraints, advanced reconstruction	Profitable Schemes, Tallest Billboard, Number of Ways to Form a Target String Given a Dictionary

SECTION 14 – 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	0/1 Knapsack (classic)	Medium	Amazon, Google, Microsoft	Direct value-maximization Knapsack	Foundational mechanics
2	Partition Equal Subset Sum	Medium	Amazon, Meta, Google, Microsoft	0/1 Knapsack existence variant	Existence-check DP
3	Target Sum	Medium	Amazon, Meta, Google	0/1 Knapsack counting variant (transformed to subset-sum)	Problem transformation + counting DP
4	Coin Change	Medium	Amazon, Meta, Google, Microsoft	Unbounded Knapsack minimization	Foundational unbounded DP
5	Coin Change II	Medium	Amazon, Google	Unbounded Knapsack counting variant	Counting-ways unbounded DP
6	Last Stone Weight II	Medium	Amazon, Google	0/1 Knapsack (transformed subset-sum minimization)	Problem transformation mastery
7	Ones and Zeroes	Medium	Amazon, Google, Meta	Multi-dimensional (dual-capacity) 0/1 Knapsack	Advanced multi-dimensional Knapsack

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
8	Profitable Schemes	Hard	Google, Amazon	Multi-dimensional Knapsack with counting	Advanced multi-dimensional counting DP
9	Rod Cutting (classic)	Medium	Amazon, Google (conceptually asked)	Unbounded Knapsack value maximization	Unbounded value-maximization mastery
10	Combination Sum IV	Medium	Amazon, Google	Unbounded Knapsack counting (order matters, differs from Coin Change II)	Order-sensitive counting DP
11	Perfect Squares (contrast, revisit from 1D DP)	Medium	Amazon, Google	Unbounded Knapsack framing (minimize count)	Cross-pattern reinforcement
12	Tallest Billboard	Hard	Google, Amazon	Advanced subset-sum-difference Knapsack variant	Advanced difference-tracking DP
13	Number of Ways to Form a Target String Given a Dictionary	Hard	Google	Sequence-position + Knapsack-style counting hybrid	Advanced hybrid DP
14	Shopping Offers	Medium	Amazon, Google	Bounded/multi-item Knapsack with special offers	Advanced bounded Knapsack variant
15	Minimum Cost For Tickets	Medium	Amazon, Google	Unbounded-style DP over a day-based capacity	Time-based unbounded DP
16	Filling Bookcase Shelves	Medium	Google	Related capacity-constrained DP (shelf width as capacity)	Capacity-constrained DP variant
17	Predict the Winner (contrast, game DP)	Medium	Google	Contrast: interval/game-theoretic DP, not Knapsack	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
18	Partition Array Into Two Arrays to Minimize Sum Difference	Hard	Google, Amazon	Advanced subset-sum-difference Knapsack	Advanced difference-minimization DP
19	Number of Dice Rolls With Target Sum	Medium	Amazon, Google	Bounded Knapsack-style counting DP	Bounded counting DP
20	Coin Change (minimum, contrast with greedy)	Medium	Amazon	Reinforce DP over greedy misconception for coin change	Greedy vs DP boundary awareness
21	Minimum Swaps To Make Sequences Increasing (contrast)	Medium	Google	Contrast: 1D DP with dual-state tracking, not Knapsack	Pattern-boundary awareness
22	Split Array into Fibonacci Sequence (contrast, backtracking)	Medium	Google	Contrast: Backtracking, not Knapsack DP	Pattern-boundary awareness
23	Matchsticks to Square (contrast, backtracking)	Medium	Google	Contrast: Backtracking with partition constraint, not pure Knapsack	Pattern-boundary awareness
24	Partition to K Equal Sum Subsets (contrast, backtracking)	Medium/Hard	Amazon, Google	Contrast: Backtracking, related to Knapsack existence-checking	Pattern-boundary awareness
25	Stone Game IV (contrast, game DP)	Hard	Google	Contrast: game-theoretic DP using perfect-square subtraction	Pattern-boundary awareness
26	Minimum Number of Refueling Stops (contrast, greedy/heap)	Hard	Google, Amazon	Contrast: Greedy + heap, not Knapsack DP	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
27	Two Sum (contrast, hashing)	Easy	Amazon	Contrast: pure hashing, included for boundary practice	Pattern-boundary awareness
28	Maximum Students Taking Exam (contrast, bitmask DP)	Hard	Google	Contrast: Bitmask DP, not classic Knapsack	Cross-pattern awareness (leads to Pattern #26)
29	Ones and Zeroes (reinforcement)	Medium	Amazon, Meta	Reinforce multi-dimensional Knapsack mastery	Multi-dimensional Knapsack reinforcement
30	Design a Budget Allocation Optimizer (custom/interview variant)	Hard	Amazon, Google (systems/business adjacent)	Applied 0/1 Knapsack for real-world budget optimization	Applied system design

SECTION 15 — Common Mistakes

1. Using the wrong loop direction when space-optimizing to 1D — forward iteration for 0/1 Knapsack (incorrectly allowing item reuse) or backward for Unbounded (incorrectly preventing reuse). *Fix:* always explicitly verify: 0/1 → capacity loop BACKWARD; Unbounded → capacity loop FORWARD.
2. Not recognizing a disguised Knapsack problem (e.g., “Partition Equal Subset Sum” or “Target Sum” don’t mention “knapsack” or “capacity” explicitly). *Fix:* look for the underlying “select a subset under a constraint” structure, regardless of surface wording.
3. Confusing “0 ways/0 value” with “target unreachable” in existence/counting variants, using inconsistent sentinel values. *Fix:* explicitly decide and initialize sentinels (e.g., infinity for “unreachable” in minimization, distinct from 0 which means “reachable with cost 0”).
4. Forgetting multi-dimensional Knapsack variants (Ones and Zeroes) need a **2D capacity** (two simultaneous constraints), requiring nested capacity loops both iterated in the correct direction. *Fix:* recognize when more than one capacity dimension exists and extend the DP state accordingly.
5. Attempting Knapsack DP when the capacity is astronomically large, without recognizing the pseudo-polynomial complexity limit. *Fix:* always sanity-check the numeric size of “capacity” against feasible DP table sizes.

Why people fail: the 0/1-vs-Unbounded loop-direction distinction is genuinely counterintuitive on first exposure and easy to get backward under time pressure, especially since both directions produce *syntactically* valid, running code — only careful reasoning about “am I allowing reuse of the item I just processed” reveals the bug, which is exactly why interviewers frequently probe this exact detail.

SECTION 16 — Optimization Techniques

- **Time:** Already pseudo-polynomial optimal at $O(n \times \text{capacity})$ for the general Knapsack formulation — no further asymptotic improvement is known for the general 0/1 case (it's NP-hard in the strict sense).
 - **Space:** Always space-optimize from $O(n \times \text{capacity})$ to $O(\text{capacity})$ using a 1D rolling array — remembering the critical backward (0/1) vs forward (Unbounded) loop direction.
 - **Readability:** Clearly comment which Knapsack variant (0/1 vs Unbounded) is being used and why the loop direction matters, since this is the single most important implementation detail to get right and communicate.
 - **Interview performance:** Explicitly verbalize the 0/1 vs Unbounded distinction and its loop-direction consequence before coding — this precise articulation is one of the strongest possible signals of genuine (not memorized) Knapsack DP understanding.
-

SECTION 17 — Multiple Language Templates

Java

```
public boolean canPartition(int[] nums) {
    int sum = Arrays.stream(nums).sum();
    if (sum % 2 != 0) return false;
    int target = sum / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int num : nums) {
        for (int w = target; w >= num; w--) {           // BACKWARD: 0/1 Knapsack
            dp[w] = dp[w] || dp[w - num];
        }
    }
    return dp[target];
}
```

JavaScript

```
function canPartition(nums) {
    const sum = nums.reduce((a, b) => a + b, 0);
    if (sum % 2 !== 0) return false;
    const target = sum / 2;
    const dp = new Array(target + 1).fill(false);
    dp[0] = true;
    for (const num of nums) {
        for (let w = target; w >= num; w--) {           // BACKWARD: 0/1 Knapsack
            dp[w] = dp[w] || dp[w - num];
        }
    }
    return dp[target];
}
```

PHP

```
function canPartition(array $nums): bool {
    $sum = array_sum($nums);
    if ($sum % 2 !== 0) return false;
```

```

$target = intdiv($sum, 2);
$dp = array_fill(0, $target + 1, false);
$dp[0] = true;
foreach ($nums as $num) {
    for ($w = $target; $w >= $num; $w--) {          // BACKWARD: 0/1 Knapsack
        $dp[$w] = $dp[$w] || $dp[$w - $num];
    }
}
return $dp[$target];
}

```

Python

```

def can_partition(nums):
    total = sum(nums)
    if total % 2 != 0:
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for w in range(target, num - 1, -1):          # BACKWARD: 0/1 Knapsack
            dp[w] = dp[w] or dp[w - num]
    return dp[target]

```

Go

```

func canPartition(nums []int) bool {
    sum := 0
    for _, n := range nums {
        sum += n
    }
    if sum%2 != 0 {
        return false
    }
    target := sum / 2
    dp := make([]bool, target+1)
    dp[0] = true
    for _, num := range nums {
        for w := target; w >= num; w-- {              // BACKWARD: 0/1 Knapsack
            dp[w] = dp[w] || dp[w-num]
        }
    }
    return dp[target]
}

```

C++

```

bool canPartition(vector<int>& nums) {
    int sum = accumulate(nums.begin(), nums.end(), 0);
    if (sum % 2 != 0) return false;
    int target = sum / 2;
    vector<bool> dp(target + 1, false);

```

```

dp[0] = true;
for (int num : nums) {
    for (int w = target; w >= num; w--) {           // BACKWARD: 0/1 Knapsack
        dp[w] = dp[w] || dp[w - num];
    }
}
return dp[target];
}

```

SECTION 18 — Dry Runs

Small Input

nums = [1, 5, 11, 5] (Partition Equal Subset Sum, target = 11)

dp = [T,F,F,F,F,F,F,F,F,F,F] (index 0..11, dp[0]=true)

num=1: w from 11 down to 1: dp[1] = dp[1] || dp[0] = true

dp=[T,T,F,F,F,F,F,F,F,F,F]

num=5: w from 11 down to 5: dp[6]=dp[6] || dp[1]=true; dp[5]=dp[5] || dp[0]=true

dp=[T,T,F,F,F,T,T,F,F,F,F]

num=11: w from 11 down to 11: dp[11]=dp[11] || dp[0]=true

dp=[T,T,F,F,F,T,T,F,F,F,T]

num=5: w from 11 down to 5: dp[11]=dp[11] || dp[6]=true(already); dp[10]=dp[10] || dp[5]=true; dp[6]=dp[6] || dp[1] becomes true

Final: dp[11] = true → CAN partition into two equal-sum subsets ([11] and [1,5,5])

Large Input (Conceptual)

For $n=200$ items with a target sum of 10^4 , the 1D space-optimized DP costs $O(200 \times 10^4) = 2 \times 10^6$ operations — fast and feasible, versus $O(2^{200})$ brute-force subset enumeration, utterly infeasible.

Corner Case

nums = [1], target sum check: total=1, odd → immediately return false without any DP computation, correctly handling the trivial impossible-partition case.

SECTION 19 — Advanced Concepts

- **Multi-dimensional Knapsack (Ones and Zeroes):** when items consume **two** separate capacities simultaneously (e.g., a string consumes both “0-budget” and “1-budget”), extend the DP state to `dp[zeros][ones]`, iterating both capacity dimensions backward (for 0/1 semantics) — a direct generalization of the single-capacity case.
- **Order-sensitive vs order-insensitive counting (Combination Sum IV vs Coin Change II):** a subtle but critical distinction — if the problem considers [1,2] and [2,1] as different “ways” (permutations matter), the capacity loop must be the OUTER loop with items as the INNER loop; if only combinations matter (order-insensitive), items must be the OUTER loop with capacity INNER — getting this backward silently produces wildly different (and wrong) counts.

- **Reconstruction of the chosen subset:** maintain a parallel table recording whether each item was included at each state, then backtrack from $dp[n][capacity]$ to recover the actual optimal subset, not just its value.
 - **Bounded Knapsack (limited reuse count):** when an item can be used up to k times (not 0/1, not unlimited), decompose it into $O(\log k)$ “binary” copies (1, 2, 4, ... up to k) and apply 0/1 Knapsack to these decomposed copies — a clever technique reducing bounded Knapsack to standard 0/1 Knapsack in $O(n \log k \times capacity)$.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Knapsack DP as the canonical formalization of **constrained resource allocation** — the same underlying structure (select items under a budget to optimize value) appears in cloud resource scheduling, portfolio optimization, and cutting-stock manufacturing problems. They’re fluent in recognizing disguised Knapsack problems (Partition Equal Subset Sum, Target Sum) that don’t mention “capacity” explicitly, and they know the pseudo-polynomial complexity caveat — that Knapsack DP is efficient for reasonably-sized numeric capacities but remains NP-hard in the strict theoretical sense for arbitrarily large capacities, a nuance worth raising proactively in Staff-level discussions. Interviewers evaluate whether a candidate can correctly and confidently articulate the 0/1-vs-Unbounded loop-direction distinction, since getting it backward is the single most common and consequential Knapsack DP bug.

SECTION 21 — Cheat Sheet

PATTERN: Knapsack DP (0/1 and Unbounded)

RECOGNIZE: "select items under capacity/budget constraint," "at most once" (0/1) vs "unlimited" (Unbounded)

TEMPLATE (0/1, space-optimized):

```
dp = [0/false] * (capacity+1); dp[0] = base value
for each item (weight, value):
    for w from capacity DOWN TO weight:           # BACKWARD prevents reuse
        dp[w] = combine(dp[w], value + dp[w-weight])
```

TEMPLATE (Unbounded, space-optimized):

```
for each item (weight, value):
    for w from weight UP TO capacity:             # FORWARD allows reuse
        dp[w] = combine(dp[w], value + dp[w-weight])
```

COMPLEXITY: $O(n \times capacity)$ - pseudo-polynomial

KEY PROOF: state (items considered, capacity remaining) collapses exponential subset space into polynomial

WATCH FOR: loop direction (backward=0/1, forward=Unbounded), order-sensitive vs insensitive counting (loop order)

DOESN'T APPLY WHEN: no capacity constraint, need ALL subsets not just optimal value (Backtracking), capacity is not numeric

SECTION 22 — Revision Notes (5-Minute Review)

- Knapsack DP state: (items considered, capacity remaining) — collapses exponential subset space to polynomial.
- 0/1 Knapsack: each item used at most once → space-optimized loop iterates capacity BACKWARD.
- Unbounded Knapsack: items reusable → space-optimized loop iterates capacity FORWARD.
- Existence variant: boolean OR combine. Counting variant: sum combine. Optimization variant: max/min combine.
- Order-sensitive counting (permutations) needs capacity as the outer loop; order-insensitive (combinations) needs items as the outer loop.

- Pseudo-polynomial complexity: $O(n \times \text{capacity})$ — becomes infeasible for astronomically large capacities.
- Many problems (Partition Equal Subset Sum, Target Sum) are disguised Knapsack — look for “select subset under constraint” structure.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Classic 0/1 Knapsack mechanics	0/1 Knapsack (GFG classic), Partition Equal Subset Sum (416)
Intermediate	Unbounded Knapsack, coin problems	Coin Change (322), Coin Change II (518)
Advanced	Counting variants, multi-dimensional	Target Sum (494), Ones and Zeroes (474), Last Stone Weight II (1049)
Expert	Advanced hybrid and bounded variants	Profitable Schemes (879), Tallest Billboard (956), Number of Dice Rolls With Target Sum (1155)

SECTION 24 — Memory Tricks

- **Mnemonic: “0/1 = Backward, Un = Forward”** (0/1 Knapsack loops the capacity Backward; Unbounded loops Forward).
 - **Visualization: Packing a suitcase with unique items (0/1) vs. shopping with unlimited stock of each item (Unbounded).**
 - **Recognition shortcut:** “Select items under a capacity/budget constraint” → Knapsack DP; “at most once” vs “unlimited” → determines loop direction.
-

SECTION 25 — Final Summary

Knapsack DP collapses the exponential “which subset of items” decision space into a polynomial (pseudo-polynomial) (items considered, capacity remaining) state space, correctly computing optimal value, existence, or count via a recurrence that either excludes or includes each item. The single most important thing to remember forever: **0/1 Knapsack (each item used at most once) requires iterating the space-optimized capacity loop BACKWARD, while Unbounded Knapsack (items reusable) requires iterating it FORWARD — getting this direction backward is the single most common Knapsack bug, silently producing wrong answers because both directions produce syntactically valid, runnable code.** Many real interview problems (Partition Equal Subset Sum, Target Sum, Coin Change) don’t mention “knapsack” or “capacity” explicitly — recognizing the underlying “select items under a constraint” structure is the key pattern-recognition skill this handbook is meant to build.